



ITI · INTRODUCTION TO PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey*Lab 5 — Getting Comfortable with NumPy Arrays*

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Lab 5 of 6
Focus	Building simple NumPy arrays, reading class scores out of them, and doing basic math and statistics — no prior NumPy experience assumed.

Before You Start

This lab keeps things simple on purpose — every exercise uses the same small, relatable dataset: a class's quiz scores. If you can create an array, look inside it, and do basic math on it by the end of this lab, you've got everything you need for Day 6.

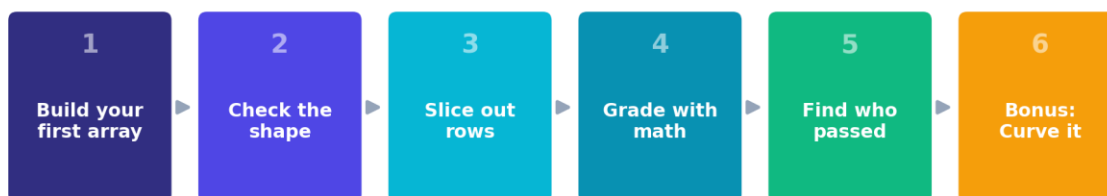


Figure L.1 — Six small steps: build an array, check its shape, slice it, do math, filter it, then curve it.



✓ QUICK CHECK — Before You Start

Q1. What line of code do you need at the top of every file that uses NumPy?

Answer: `import numpy as np`

Q2. What function turns a plain Python list into a NumPy array?

Answer: `np.array(your_list)`

Q3. What does `.shape` tell you about an array?

Answer: The size of each dimension, e.g. `(3, 4)` means 3 rows and 4 columns.



01

Build Your First Array

A teacher recorded five quiz scores for one student: 78, 85, 92, 60, 88. Turn this into a NumPy array and print it, along with two of its basic properties.

- Start your file with `import numpy as np`.
- Store the five scores in a plain Python list first, then convert it with `np.array()`.
- Print the array itself, then print its `.shape` and its `.dtype`.

AI connection: Almost every dataset you will ever load for an AI project ends up as a NumPy array (or something built on top of one) — this is the very first step of that journey.

Expected output:

```
[78 85 92 60 88]
(5,)
int64
```

02

A Whole Class of Scores

Real classes have more than one student. Build a 2D array holding scores for three students, each with four quiz scores, then explore its shape.

- Create a 2D array (a list of lists) for 3 students $\times$ 4 quizzes — you can invent the numbers.
- Print the array, then print `.shape` and `.ndim`.
- Print `grid[0]` to see the first student's row, and `grid[0, 2]` to see just their third quiz score.

AI connection: This is exactly the shape (rows = examples, columns = features) that almost every dataset takes before it's handed to a machine learning model.

Expected output:

```
[[90 85 78 92]
 [70 88 95 60]
 [82 77 91 84]]
(3, 4)
2
[90 85 78 92]
78
```



03

Slice Out Rows and Columns

Using the same `quiz_grid` from Exercise 02, practice pulling out specific pieces of the grid using slicing.

- Print the first two students only, using `quiz_grid[0:2]`.
- Print every student's score on the first quiz only, using `quiz_grid[:, 0]`.
- Print the last student's last quiz score using negative indices: `quiz_grid[-1, -1]`.

AI connection: Slicing out "just this column" or "just these rows" is exactly how you'd pull out a single feature (like age) or a handful of examples from a much larger dataset.

Expected output:

```
[[90 85 78 92]
 [70 88 95 60]]
[90 70 82]
84
```

04

Grade with Math

The teacher decides every student deserves a 5-point curve on quiz 3 (it was a hard one), and wants to know the average and highest score per student. Do this with array math instead of a loop.

- Add 5 only to the third column (quiz 3) using `quiz_grid[:, 2] = quiz_grid[:, 2] + 5`.
- Print the updated grid to confirm the curve applied.
- Print `quiz_grid.mean(axis=1)` for each student's average, and `quiz_grid.max(axis=1)` for each student's best quiz.

AI connection: Adjusting one column of a whole dataset in a single line, with no loop, is the everyday reality of cleaning and adjusting data before training a model.

Expected output:

```
[ [ 90  85  83  92]
  [ 70  88 100  60]
  [ 82  77  96  84]]
[87.5 79.5 84.75]
[ 92 100  96]
```

05

Find Who Passed

Using each student's average from Exercise 04, find out who passed (average of 80 or above) using boolean indexing instead of an if statement inside a loop.

- Store the averages in a variable: `averages = quiz_grid.mean(axis=1)`.
- Build a True/False array with `passed = averages >= 80`, and print it.
- Print `averages[passed]` to see only the passing averages.
- Print how many students passed using `passed.sum()` (True counts as 1, False as 0).

AI connection: Filtering rows by a condition like this is one of the most common things you'll do once you move on to pandas — the exact same `>=` comparison and boolean-indexing idea carries over.

Expected output:

```
[ True False  True]
[87.5  84.75]
Number who passed: 2
```